

Tutorial 5: AI-Assisted R Programming for Financial Modeling

(STUDENT VERSION)

Theme: Asset Comparison and Portfolio Synergy in R

Materials created under GE Fund OSALP – for authorised educational use only.

Programme: Nurturing Future Risk Managers and Actuaries: An AI-powered Journey for Gifted Secondary Students

Synergy & Comparison

The Logic of this Tutorial: A skilled actuary doesn't just calculate prices; they compare behaviors and quantify risk constraints. In this tutorial, we will retrieve historical data to compare the raw volatility and maximum drawdowns of stocks vs. bonds (Section A). We will then build mathematical models to explain *why* bonds are anchored by convexity (Section B) and *why* stocks are volatile to growth shocks (Section C). Finally, we synergize these opposites into a single portfolio, proving mathematically how adjusting weights optimizes the risk-return profile (Section D).

1 A. Observing the Market: Equity vs. Debt

Key Definition / Result

Before modeling theory, we observe reality. Using the `quantmod` and `PerformanceAnalytics` packages, we can pull real-world pricing data for both the S&P 500 (Equity/Stocks, ticker: `SPY`) and the US Aggregate Bond Index (Debt/Bonds, ticker: `AGG`) to visually and statistically compare their historical risk.

Suggested AI Prompt: Comparative Visualisation & Tail Risk ★ Core

"Act as an R programming mentor. I need an R script using 'quantmod' to download the last 5 years of daily adjusted closing prices for SPY and AGG. Plot their normalized prices on the same chart starting at 100. Then, use 'Perfor-

manceAnalytics' to calculate the Maximum Drawdown for both assets. Please comment the code clearly."

Worked Example: R Code for Comparative Prices & Drawdowns ★

Core

R Code:

```
library(quantmod)
library(PerformanceAnalytics)

getSymbols(c("SPY", "AGG"), src = "yahoo", from = Sys.Date() -
  365*5)

# Extract Adjusted Close and Normalize to start at 100
spy_norm <- (Ad(SPY) / as.numeric(Ad(SPY)[1])) * 100
agg_norm <- (Ad(AGG) / as.numeric(Ad(AGG)[1])) * 100

# Plot together for visual comparison
plot(spy_norm, main = "Synergy Observation: SPY vs AGG", col="
  blue", ylim=c(70, 250))
lines(agg_norm, col="red", lwd=2)
legend("topleft", legend=c("SPY (Equity)", "AGG (Bonds)"), col=
  c("blue", "red"), lty=1)

# Calculate Daily Returns & Max Drawdown
spy_ret <- Return.calculate(Ad(SPY))
agg_ret <- Return.calculate(Ad(AGG))

cat("SPY Max Drawdown:", maxDrawdown(spy_ret) * 100, "%\n")
cat("AGG Max Drawdown:", maxDrawdown(agg_ret) * 100, "%\n")
```

Suggested AI Prompt: Rolling Volatility Analysis

★★ Intermediate

"I have daily log returns for SPY and AGG stored in R. I want to calculate and plot a 30-day rolling annualized standard deviation (volatility) to see how market risk changes over time. How do I do this using the `chart.RollingPerformance`

function from the `PerformanceAnalytics` package? What does the 'scale' parameter do?"

Investigation / Activity: Rolling Volatility Analysis ★★ Intermediate

Concept Scaffold: Lecture 5 introduces financial change via returns. **Volatility** builds on this by measuring the standard deviation (spread) of those returns. A **rolling volatility** tracks this risk over a moving window (e.g., 30 days), revealing how risk itself fluctuates over time.

Task. Extend the previous script to calculate and plot the 30-day rolling volatility for both SPY and AGG.

R Code:

```
# Calculate 30-day rolling standard deviation (volatility)
chart.RollingPerformance(R = cbind(spy_ret, agg_ret),
                          width = 30,
                          FUN = "sd.annualized",
                          scale = 252,
                          main = "30-Day Rolling Volatility: SPY
                                vs AGG",
                          colorset = c("blue", "red"),
                          legend.loc = "topleft")
```

2 B. The Anchor: Bond Valuation and Convexity

Synergy & Comparison

Why was the red line (AGG) so stable in Section A? Lecture 5 shows that bond prices are the present value of future cash flows. Because the market yield is in the denominator ($1/(1+y)^t$), the price-yield relationship is not linear. We call this curvature **Convexity**—the phenomenon where bond prices rise faster when yields drop than they fall when yields rise. Let's model this in R.

Suggested AI Prompt: Simulating Bond Convexity**** Insight**

"I am learning about bond valuation. Please help me write an R function that calculates the present value of a bond given its face value, coupon rate, market yield, and years to maturity. Then, show me how to use a sequence to calculate the price of a 10-year, 5% coupon bond for market yields ranging from 1% to 15%, and plot the resulting convexity curve."

Investigation / Activity: Plotting Yield-to-Price Convexity**** Insight**

Task. Create an R function for bond valuation. Run a loop/sequence to calculate prices across a wide range of market yields. Plot the relationship.

R Code:

```
price_bond <- function(face_value, coupon_rate, market_yield,
  years) {
  t <- 1:years
  pv_coupons <- sum((face_value * coupon_rate) / (1 + market_
    yield)^t)
  pv_face <- face_value / (1 + market_yield)^years
  return(pv_coupons + pv_face)
}

# Evaluate price sensitivity to a wide range of yields
yields <- seq(0.01, 0.15, by=0.005)
prices <- sapply(yields, function(y) price_bond(1000, 0.05, y,
  10))

# Plot the Convexity Curve
plot(yields * 100, prices, type="l", col="red", lwd=3,
  main="Bond Convexity: Price vs. Market Yield",
  xlab="Market Yield (%)", ylab="Bond Price ($)")
grid()
```

Suggested AI Prompt: Code Optimization: Vectorization********Optimization**

*"My current bond pricing code uses **sapply** to loop through yields. How can I rewrite this R function using **vectorized operations** so I can calculate millions of bond prices instantly without using loops or apply functions?"*

3 C. The Engine: Stock Valuation and Growth Shocks

Synergy & Comparison

Why was the blue line (SPY) so erratic in Section A? Lecture 5 introduced the constant-growth stock valuation model ($P_0 = D_1/(k - g)$). In reality, companies often have an initial period of high growth before settling into a stable rate. To model this, we use a **Two-Stage Dividend Discount Model**, which calculates the discrete present value for the high-growth years and uses the constant-growth formula for the stable terminal value.

Suggested AI Prompt: Building the Two-Stage DDM ★★★ Advanced

"Act as a financial mathematics tutor. I need to build a Two-Stage Dividend Discount Model in R. Write a function that takes the current dividend (D_0), required return (k), high growth rate (g_high), stable growth rate (g_stable), and the number of high growth years ($years_high$). Explain step-by-step how the present value of stage 1 and the terminal value are calculated."

Worked Example: Two-Stage Growth Sensitivity ★★★ Advanced

Question. Use the generated R function for a Two-Stage Growth Model. Compare how a 2% shock to the short-term growth rate compares to a 2% shock in the required return (k).

R Code:

```
price_stock_2stage <- function(D0, k, g_high, g_stable, years_
  high) {
  t <- 1:years_high

  # Stage 1: High Growth PV
```

```

divs_stage1 <- D0 * (1 + g_high)^t
pv_stage1 <- sum(divs_stage1 / (1 + k)^t)

# Stage 2: Terminal Value PV
D_term <- divs_stage1[years_high] * (1 + g_stable)
TV <- D_term / (k - g_stable)
pv_TV <- TV / (1 + k)^years_high

# Final Price
return(pv_stage1 + pv_TV)
}

# Run Scenarios
base_price <- price_stock_2stage(D0=2, k=0.08, g_high=0.12, g
  _stable=0.03, years_high=5)
growth_shock <- price_stock_2stage(D0=2, k=0.08, g_high=0.10, g
  _stable=0.03, years_high=5)
risk_shock <- price_stock_2stage(D0=2, k=0.10, g_high=0.12, g
  _stable=0.03, years_high=5)

# Print Results
cat("Base Price:   $", round(base_price, 2), "\n")
cat("Growth Shock: $", round(growth_shock, 2), "\n")
cat("Risk Shock:   $", round(risk_shock, 2), "\n")

```

4 D. The Synergy: Portfolio Construction and the Efficient Frontier

Synergy & Comparison

The Ultimate Actuarial Goal: Lecture 5 discusses choosing assets to match liabilities. However, actuaries rarely hold just one asset; they combine them into portfolios. Because assets do not move in perfect sync, combining them can mathematically reduce total risk—a concept tied to the **Efficient Frontier**. To measure how efficiently a portfolio balances risk and reward, we use the **Sharpe**

Ratio, defined as the portfolio's return minus the risk-free rate, all divided by the portfolio's volatility.

Suggested AI Prompt: Simulating the Efficient Frontier *** Advanced

"I have historical daily log returns for SPY and AGG. I want to build a loop in R that tests portfolio allocations from 0% SPY up to 100% SPY (in 1% increments). For each combination, how do I calculate the annualized portfolio volatility and annualized expected return? Then, show me how to calculate the Sharpe Ratio assuming a 2% risk-free rate, and find the exact weight that maximizes it."

Investigation / Activity: Sweeping Weights to Optimize Synergy *** Advanced

Task. Test portfolio allocations from 0% SPY to 100% SPY. Plot the Volatility against the Weights to find the "Minimum Variance Portfolio" (the combination with the lowest possible risk).

R Code:

```
spy_ret <- diff(log(Ad(SPY)))[-1]
agg_ret <- diff(log(Ad(AGG)))[-1]
weights_spy <- seq(0, 1, by=0.01)

port_vols <- numeric(length(weights_spy))
port_ret <- numeric(length(weights_spy))

# Loop through combinations
for(i in 1:length(weights_spy)) {
  w_s <- weights_spy[i]
  w_a <- 1 - w_s
  port_ret <- (w_s * spy_ret) + (w_a * agg_ret)

  port_vols[i] <- sd(port_ret, na.rm=TRUE) * sqrt(252) * 100
  port_ret[i] <- mean(port_ret, na.rm=TRUE) * 252 * 100
}

# Plot Risk Profile (Volatility vs. SPY Weight)
```

```

plot(weights_spy * 100, port_vols, type="l", col="purple", lwd
     =2,
      xlab="Weight of SPY (%)", ylab="Annualized Volatility (%)",
      ,
      main="Portfolio Risk Curve")

# Find the Minimum Variance Portfolio
min_vol_index <- which.min(port_vols)
opt_spy_weight <- weights_spy[min_vol_index] * 100
cat("The lowest risk is achieved with", opt_spy_weight, "% SPY
    .\n")

```

Investigation / Activity: Maximizing the Sharpe Ratio *** Advanced

Task. Extend the loop from the previous example to calculate the Sharpe Ratio for each portfolio weight combination, assuming an annual risk-free rate of 2%. Find the weight of SPY that maximizes the Sharpe Ratio.

R Code:

```

risk_free_rate <- 0.02
# Calculate Sharpe Ratio for each portfolio combination
port_sharpes <- (port_rets/100 - risk_free_rate) / (port_vols/
    100)

# Plot Sharpe Ratio vs Weight of SPY
plot(weights_spy * 100, port_sharpes, type="l", col="darkgreen",
     , lwd=2,
      xlab="Weight of SPY (%)", ylab="Sharpe Ratio",
      main="Portfolio Efficiency Curve")

# Find the Maximum Sharpe Ratio Portfolio
max_sharpe_index <- which.max(port_sharpes)
opt_sharpe_spy_weight <- weights_spy[max_sharpe_index] * 100
cat("The optimal Sharpe Ratio is achieved with", opt_sharpe_spy
    _weight, "% SPY.\n")

```